

Bidirectional Cognitive Stream Fatigue Networks for Human-Like Parallel Cognitive Processing

Anonymous Authors

Preprint — April 2026

ABSTRACT

Human cognition involves at least two simultaneous processes: a slow, deep *inner stream* of ongoing reasoning and a fast *expressive stream* of surface-level output. These processes run in parallel, influence each other bidirectionally, and are regulated by synaptic fatigue — the tendency of frequently active neurons to become transiently less responsive, naturally rotating representational load across the network. Existing recurrent architectures conflate these processes into a single hidden state, creating a representational bottleneck; Transformers eliminate dynamic state entirely. We introduce the **Cognitive Stream Fatigue Network (CSFN)**, a recurrent architecture with two bidirectionally coupled GRU streams — an inner (conceptual) stream Ψ and an expressive stream Λ — together with a neural fatigue accumulator ϕ per stream and two independent learned confidence gates that mediate cross-stream communication in both directions. Fatigue is modelled as an exponentially decaying activation trace that suppresses recently over-active neurons, forcing representational diversity over time. We derive the full parameter count, state update equations, and a PyTorch reference implementation. We design three evaluation protocols: a delayed copy task at multiple delays, character-level language modelling on the Penn Treebank, and a novel dual-context tracking task that directly tests simultaneous maintenance of two independent signal sources. Ablation studies isolate the contribution of each mechanism. All baselines are parameter-matched and evaluated over five random seeds.

Keywords: recurrent neural networks, multi-timescale learning, neural fatigue, bidirectional coupling, cognitive architectures, parallel processing, sequence modelling

1. Introduction

Consider the experience of formulating a sentence while simultaneously holding a longer train of thought. The words being spoken are determined by a fast, surface-level process that must track grammar, prosody, and immediate context. Running in parallel, a slower process maintains the semantic thread — the *reason* for the sentence, its relation to what came before, and what should follow. These two processes are not independent: a shift in the background reasoning can redirect mid-sentence expression, and what gets said can in turn update the underlying thought. Neither stream is ever fully idle, and neither is a simple bottleneck for the other.

Biological neural circuits implement this through several mechanisms absent from current artificial architectures. First, neurons are connected in all directions — fast surface representations send signals back to deeper layers just as readily as they receive top-down guidance. Second, neurons exhibit **synaptic fatigue**: repeated activation depletes neurotransmitter reserves, transiently reducing responsiveness. This fatigue is not a failure mode; it is a functional mechanism that prevents any one representational pattern from monopolising a network indefinitely, naturally rotating cognitive load and enabling new patterns to emerge [13].

Existing sequence models miss both properties. Transformers [1] generate tokens via feedforward passes over a static key-value cache — there is no dynamic hidden state whatsoever. Recurrent models (GRU [3], LSTM [2]) maintain a single dynamic state that must simultaneously represent fast-changing and slow-drifting signal components, creating a representational bottleneck. State-space models such as Mamba [5] and S4 [6] introduce selective memory retention but operate on a single latent timescale with no fatigue or bidirectional coupling. Recent recurrent revivals — xLSTM [14], Griffin [15], RWKV [16] — improve expressivity and throughput but retain the single-state paradigm and add no fatigue

mechanism.

We propose the **Cognitive Stream Fatigue Network (CSFN)**, which addresses all three limitations through four complementary mechanisms: (i) two parallel GRU streams, Ψ (inner / conceptual) and Λ (expressive / surface), operating at structurally distinct timescales; (ii) **bidirectional confidence gates** that mediate information flow from Ψ to Λ and from Λ to Ψ independently; (iii) a **neural fatigue accumulator** per stream that suppresses over-active neurons and forces representational diversity; and (iv) independent input-conditioned adaptive decay MLPs for each stream. Together these mechanisms produce a continuously evolving, self-regulating internal state that better reflects the parallel structure of biological cognition.

The remainder of the paper is organised as follows. Section 2 situates CSFN within the literature. Section 3 derives the full architecture. Section 4 provides a cognitive interpretation of each mechanism. Section 5 details the experimental protocol with parameter-matched baselines. Section 6 presents results and analysis. Sections 7 and 8 discuss limitations and conclude.

2. Related Work

2.1 Transformer Models

GPT-style autoregressive Transformers [1, 7] generate tokens by attending over all previous positions via causal self-attention. The key-value cache stores past representations but is never rewritten in response to new tokens — it grows monotonically. Recurrent Transformers such as Transformer-XL [8] reintroduce a segment memory, but it remains a fixed snapshot rather than a continuously updated hidden state. CSFN differs by maintaining a dynamic, gated, fatigue-modulated hidden state updated at every timestep, with constant memory cost.

2.2 LSTM and GRU

The Long Short-Term Memory [2] introduces input, forget, and output gates to control information flow into a cell state. The Gated Recurrent Unit [3] simplifies this to two gates with comparable empirical performance. Both maintain a *single* hidden state that must represent all temporal horizons simultaneously. CSFN decomposes the hidden state into two streams with structurally enforced timescale separation, adds bidirectional cross-stream gates absent in both, and introduces the fatigue accumulator as a third regulatory layer.

2.3 xLSTM, Griffin, RWKV, and RetNet

Recent recurrent revivals improve on LSTM and GRU along different axes. xLSTM [14] introduces exponential gating and multi-head memory matrices, achieving competitive language modelling performance. Griffin [15] (Google DeepMind) combines linear recurrence with local attention in a hybrid architecture. RWKV [16] reformulates attention as a linear recurrence enabling transformer-quality results with linear inference cost. RetNet [17] introduces explicit multi-scale retention heads that decay at fixed geometric rates. None of these architectures model synaptic fatigue or implement bidirectional coupling between explicitly separated timescale streams.

2.4 Hierarchical and Clock-Driven RNNs

Hierarchical Multiscale RNNs [9] organise recurrent layers into a temporal hierarchy where lower layers update more frequently. Crucially, [9] uses *learned* boundary detectors to control update frequency — it is not a fixed-clock model. The Clockwork RNN [10] assigns fixed update periods to neuron groups. CSFN shares the timescale decomposition intuition but implements it within a single layer via independent adaptive decay MLPs rather than by stacking layers or assigning fixed clock rates, and couples the streams bidirectionally through separate confidence gates.

2.5 State-Space Models

Structured State Space Models [6] parameterise recurrence via HiPPO-initialised state matrices. Mamba [5] adds input-dependent transition matrices (selective SSMS) and achieves linear per-step cost $O(H)$ versus the $O(H^2)$ of GRU-based models. CSFN is less parameter-efficient than Mamba but offers explicit stream interpretability and the fatigue mechanism. Mamba's Δ (discretisation step) is input-conditioned decay — analogous to CSFN's adaptive α — but applies to a single structured state without timescale decomposition or fatigue.

2.6 Predictive Coding and Biological Fatigue

Predictive coding [11] models cortical processing as a hierarchy of prediction-error signals. PredNet [12] implements this for video prediction. CSFN borrows the bidirectional feedback intuition but replaces prediction-error with a learned confidence gate applicable to arbitrary tasks. The neural fatigue mechanism is grounded in synaptic short-term depression [13]: sustained presynaptic firing depletes vesicle pools, reducing postsynaptic response amplitude — precisely the dynamic we model in ϕ .

3. Architecture

3.1 Overview and Notation

CSFN maintains two hidden states: h_Λ (expressive stream) and h_Ψ (inner / conceptual stream), each of dimension H . Two fatigue accumulators $\phi_\Lambda, \phi_\Psi \in [0,1]^H$ modulate effective activation. Two independent confidence gates $g_{\Psi \rightarrow \Lambda}$ and $g_{\Lambda \rightarrow \Psi}$ mediate bidirectional cross-stream coupling. At each timestep t the model receives an input $x_t \in \mathbb{R}^d$, updates both streams, applies fatigue, and produces output y_t .

3.2 Input Projection

$$x_t^\Lambda = W_{in} x_t + b_{in}, \quad W_{in} \in \mathbb{R}^{H \times d}$$

3.3 Neural Fatigue Accumulator

The fatigue state $\phi_{s,t}$ for stream $s \in \{\Lambda, \Psi\}$ accumulates with neuron activation and decays exponentially between timesteps:

$$\phi_{s,t} = \gamma_s \phi_{s,t-1} + (1 - \gamma_s) |h_{s,t}|$$

where $\gamma_s = \sigma(w_{\gamma,s}) \in (0,1)$ is a learned per-neuron fatigue recovery rate. A neuron with high $\gamma_{s,i}$ recovers slowly (persistent fatigue); one with low $\gamma_{s,i}$ recovers quickly (transient fatigue). The **effective** (fatigue-modulated) hidden state used in all downstream computations is:

$$h_{s,t}^\Lambda = h_{s,t} (1 - \phi_{s,t})$$

Neurons that have been consistently active accumulate high ϕ , reducing h^Λ toward zero. This prevents any representation from monopolising the network and forces activation diversity over time. Neurons that were suppressed (low $|h|$) recover quickly, making them available for new patterns. Note that ϕ is updated *after* the GRU step so that $h_{s,t}$ is computed from the undamped state and the fatigue reflects the step just taken.

3.4 Bidirectional Confidence Gates

Two independent gates mediate cross-stream communication in both directions. Each gate is computed from the fatigue-modulated joint state, so streams that are currently fatigued exert less cross-stream influence:

$$g_{\Psi \rightarrow \Lambda, t} = \sigma(W_{g1} [h_{\Psi, t-1}^\Lambda ; h_{\Lambda, t-1}^\Psi] + b_{g1}) \text{ (inner guides expression)}$$

$$g_{\Lambda \rightarrow \Psi, t} = \sigma(W_{g2} [h_{\Lambda, t-1}^\Psi ; h_{\Psi, t-1}^\Lambda] + b_{g2}) \text{ (expression feeds back into thought)}$$

The argument order is reversed between the two gates so each gate has a distinct projection of the joint state. This asymmetry allows $g_{\Psi \rightarrow \Lambda}$ and $g_{\Lambda \rightarrow \Psi}$ to learn different gating policies with the same parameter budget.

3.5 Independent Adaptive Decay

Each stream has its own input-conditioned decay MLP, giving the two streams independently adaptive forgetting rates rather than sharing a single decay signal:

$$\alpha_{\Lambda, t} = \sigma(\text{MLP}_\Lambda(x_t)) \in (0,1)^H \text{ (expressive – fast decay)}$$

$$\alpha_{\Psi, t} = \sigma(\text{MLP}_\Psi(x_t))^\beta \in (0,1)^H \text{ (inner – slow decay, } \beta = 0.2)$$

The exponent $\beta < 1$ structurally enforces that the inner stream's decay values are closer to 1 (slower forgetting) for any given MLP output. Because the two decay MLPs are independent, the expressive stream can accelerate its forgetting in response to a signal change without forcing the same on the inner stream.

3.6 Feedback Terms

$$f_{\Lambda, t} = W_{fb, \Lambda} h_{\Lambda, t-1}^\Lambda \alpha_{\Lambda, t} g_{\Psi \rightarrow \Lambda, t}$$

$$f_{\Psi, t} = W_{fb, \Psi} h_{\Psi, t-1}^\Psi \alpha_{\Psi, t} g_{\Lambda \rightarrow \Psi, t}$$

Note that each stream's feedback is modulated by the gate coming *from* the other stream. The inner stream gates expression ($\Psi \rightarrow \Lambda$), and expression gates inner feedback ($\Lambda \rightarrow \Psi$). Fatigue enters feedback through the $h_{\Lambda,t}$ terms: a fatigued inner stream automatically sends weaker guidance to the expressive stream, preventing stale background representations from dominating.

3.7 Stream Updates

$$\begin{aligned} h_{\Lambda,t} &= \text{GRUCell}_{\Lambda} (x_t + f_{\Lambda,t} + w_{\Psi \rightarrow \Lambda} h_{\Psi,t-1}, h_{\Lambda,t-1}) \\ h_{\Psi,t} &= \text{GRUCell}_{\Psi} (x_t + f_{\Psi,t} + w_{\Lambda \rightarrow \Psi} h_{\Lambda,t}, h_{\Psi,t-1}) \end{aligned}$$

The expressive stream Λ is updated first, using the *previous* inner state $h_{\Psi,t-1}$. The inner stream Ψ is then updated using the *current* expressive state $h_{\Lambda,t}$ (freshly computed). This ordering models the asymmetry in human cognition: background reasoning integrates what has just been said before updating, while surface expression is driven primarily by the previous thought state.

3.8 Fatigue Update

After both GRU updates, the fatigue accumulators are refreshed (see Section 3.3):

$$\begin{aligned} \phi_{\Lambda,t} &= \gamma_{\Lambda} \phi_{\Lambda,t-1} + (1 - \gamma_{\Lambda}) |h_{\Lambda,t}| \\ \phi_{\Psi,t} &= \gamma_{\Psi} \phi_{\Psi,t-1} + (1 - \gamma_{\Psi}) |h_{\Psi,t}| \end{aligned}$$

3.9 Readout

$$y_t = w_{\text{out}} [h_{\Lambda,t}; h_{\Psi,t}] + b_{\text{out}}$$

The readout uses exclusively fatigue-modulated states. Neurons that have been over-active cannot dominate the output regardless of their raw magnitude in h .

3.10 Parameter Count

Component	Parameters
Input projection	$H \times d + H$
Feedback projections ($\times 2$)	$2 \times (H^2 + H)$
Confidence gates $g_{\{\Psi \rightarrow \Lambda\}}$ and $g_{\{\Lambda \rightarrow \Psi\}}$ ($\times 2$)	$2 \times (H \times 2H + H)$
Adaptive decay MLPs ($\times 2$ independent)	$2 \times (Hd + H^2 + 2H)$
GRU cells ($\times 2$)	$2 \times 3(2H^2 + H)$
Fatigue recovery rates $\gamma_{\Lambda}, \gamma_{\Psi}$ ($\times 2$)	$2H$
Readout	$d_{\text{out}} \times 2H + d_{\text{out}}$
Total (approx.)	$\sim 18H^2 + (3d + d_{\text{out}} + 8)H$

Table 1: Parameter breakdown of CSFN. For $H=64, d=10, d_{\text{out}}=10$: $\approx 74,200$ parameters.

The increased parameter count relative to MSGFN ($\sim 57K$ at $H=64$) is driven by the second independent decay MLP and the second confidence gate. All baselines in experiments are parameter-matched by adjusting their hidden dimension H accordingly.

4. Cognitive Interpretation

4.1 Inner and Expressive Streams

The **expressive stream** Λ models surface-level cognitive output: the rapid, high-frequency tracking of input that drives moment-to-moment responses. In language generation this corresponds to the articulatory or phonological loop — the "inner voice" shaping each word choice. Its fast adaptive decay means it forgets quickly, staying responsive to recent input.

The **inner stream** Ψ models background reasoning: the slower accumulation of semantic and contextual meaning that gives expression its coherence over longer horizons. Its decay is structurally constrained toward 1 (slow forgetting), allowing it to maintain a train of thought across many expressive steps. Crucially, Ψ is not directly output — it

influences y_t only through $h_{\psi,t}$ in the readout and through its gate $g_{\psi \rightarrow \Lambda}$ on expression.

4.2 Neural Fatigue as Representational Regulation

Synaptic fatigue in biological networks arises from the depletion of readily releasable neurotransmitter vesicles during sustained activation [13]. The effect is a transient reduction in synaptic efficacy that recovers at a timescale governed by vesicle replenishment rates — ranging from tens of milliseconds to several seconds depending on synapse type. CSFN's fatigue accumulator ϕ directly models this: the exponential decay of ϕ corresponds to recovery, and the rate $\gamma_{s,i}$ corresponds to the neuron-specific replenishment timescale, learned from data.

Functionally, fatigue prevents *fixation*: the network cannot sustain the same representational pattern indefinitely. When one set of neurons has been driving the output for several steps, their fatigue rises, h_{ψ} falls, and the gate and readout naturally shift weight toward neurons that have been recovering. This creates an implicit attention-like rotation over the hidden population without an explicit attention mechanism.

4.3 Bidirectional Gates as Cognitive Coherence Checks

The confidence gate in MSGFN was a single shared gate that suppressed feedback when the two streams diverged. CSFN separates this into two directional gates with a distinct interpretation. $g_{\psi \rightarrow \Lambda}$ answers: "how much should the current background reasoning guide expression?" — it should be high when the inner stream has a coherent representation and low when it is uncertain or in flux. $g_{\Lambda \rightarrow \psi}$ answers: "how much should what is being said update the underlying thought?" — it should be high when the expressive output carries new semantic content and low when it is routine or predictable. Learning these policies from data allows the model to discover when to let thoughts guide words and when to let words update thoughts.

5. Comparison with Existing Architectures

Property	Transformer	GRU/LSTM	Mamba	xLSTM [14]	Griffin [15]	CSFN (ours)
Dynamic hidden state	No	Yes	Yes	Yes	Yes	Yes
Multi-timescale	No	No	No	No	Partial	Yes (2 streams)
Bidirectional coupling	No	No	No	No	No	Yes (2 gates)
Neural fatigue	No	No	No	No	No	Yes (ϕ per stream)
Input-cond. decay	N/A	No	Yes (Δ)	Partial	Yes	Yes (independent)
Streaming inference	Approx.	Yes	Yes	Yes	Yes	Yes
Cost per step	$O(n^2)$	$O(H^2)$	$O(H)$	$O(H^2)$	$O(H)$	$O(H^2)$
Interpretable timescales	No	No	No	No	No	Yes
Fatigue-based diversity	No	No	No	No	No	Yes

Table 2: Qualitative comparison. CSFN is the only architecture with bidirectional coupling, neural fatigue, and independent multi-timescale adaptive decay.

The key differentiator between CSFN and all listed architectures is the *combination* of fatigue and bidirectional coupling. Griffin [15] implements partial multi-timescale behaviour via its hybrid recurrence-attention design, but does not expose interpretable stream separation. xLSTM's exponential gating provides stronger memory than standard LSTM but remains a single state. Mamba's Δ parameter is functionally similar to CSFN's α , but it operates on a single structured state without stream decomposition or fatigue.

6. Reference Implementation

A compact PyTorch implementation is provided for reproducibility. All components are standard `nn.Module` subclasses; no custom CUDA kernels are required.

```

class CSFN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size, beta=0.2):
        super().__init__()
        H = hidden_size
        self.beta = beta
        self.input_proj = nn.Linear(input_size, H)
        # Independent decay MLPs per stream
        self.decay_lambda = nn.Sequential(nn.Linear(input_size, H), nn.ReLU(),
                                           nn.Linear(H, H), nn.Sigmoid())
        self.decay_psi = nn.Sequential(nn.Linear(input_size, H), nn.ReLU(),
                                       nn.Linear(H, H), nn.Sigmoid())
        # Feedback projections
        self.fb_lambda = nn.Linear(H, H)
        self.fb_psi = nn.Linear(H, H)
        # Bidirectional confidence gates
        self.gate_psi2lam = nn.Sequential(nn.Linear(2*H, H), nn.Sigmoid())
        self.gate_lam2psi = nn.Sequential(nn.Linear(2*H, H), nn.Sigmoid())
        # Cross-stream projection weights
        self.W_psi2lam = nn.Linear(H, H, bias=False)
        self.W_lam2psi = nn.Linear(H, H, bias=False)
        # GRU cells
        self.gru_lam = nn.GRUCell(H, H)
        self.gru_psi = nn.GRUCell(H, H)
        # Fatigue recovery rates (per neuron, per stream)
        self.gamma_lam = nn.Parameter(torch.zeros(H))
        self.gamma_psi = nn.Parameter(torch.zeros(H))
        self.readout = nn.Linear(2*H, output_size)

    def forward(self, x, h_lam, h_psi, phi_lam, phi_psi):
        # Input projection
        xp = self.input_proj(x)
        # Fatigue recovery rates
        g_lam = torch.sigmoid(self.gamma_lam)
        g_psi = torch.sigmoid(self.gamma_psi)
        # Effective (fatigue-modulated) hidden states
        h_lam_eff = h_lam * (1 - phi_lam)
        h_psi_eff = h_psi * (1 - phi_psi)
        # Adaptive decay
        alpha_lam = self.decay_lambda(x)
        alpha_psi = self.decay_psi(x) ** self.beta
        # Bidirectional gates
        joint_psi_lam = torch.cat([h_psi_eff, h_lam_eff], dim=-1)
        joint_lam_psi = torch.cat([h_lam_eff, h_psi_eff], dim=-1)
        g_p2l = self.gate_psi2lam(joint_psi_lam)
        g_l2p = self.gate_lam2psi(joint_lam_psi)
        # Feedback
        fb_lam = self.fb_lambda(h_lam_eff) * alpha_lam * g_p2l
        fb_psi = self.fb_psi(h_psi_eff) * alpha_psi * g_l2p
        # Stream updates:  $\Lambda$  first, then  $\Psi$  uses fresh  $h_{\text{lam}}$ 
        h_lam = self.gru_lam(xp + fb_lam + self.W_psi2lam(h_psi_eff), h_lam)
        h_lam_eff2 = h_lam * (1 - phi_lam) # tentative for  $\Psi$  input
        h_psi = self.gru_psi(xp + fb_psi + self.W_lam2psi(h_lam_eff2), h_psi)
        # Fatigue update
        phi_lam = g_lam * phi_lam + (1 - g_lam) * h_lam.abs()
        phi_psi = g_psi * phi_psi + (1 - g_psi) * h_psi.abs()
        # Final effective states and readout
        h_lam_eff = h_lam * (1 - phi_lam)
        h_psi_eff = h_psi * (1 - phi_psi)
        y = self.readout(torch.cat([h_lam_eff, h_psi_eff], dim=-1))
        return h_lam, h_psi, phi_lam, phi_psi, y

```

7. Experiments

7.1 Baselines and Ablations

All baselines are parameter-matched to CSFN (H=64 per stream, $\approx 74K$ parameters). GRU and LSTM hidden dimensions are set to H=100 to match. Ablations are evaluated at the same parameter budget as full CSFN (H=64).

Model	Description	$\sim$ Params
GRU (matched)	Single GRUCell, H=100	74K
LSTM (matched)	Single LSTMCell, H=90	74K
Mamba-tiny	Selective SSM, $d_{\text{model}}=64$	$\sim 74K$
xLSTM-small	Exponential-gated LSTM, H=80	$\sim 74K$
CSFN (no fatigue)	$\phi=0$ always; γ removed	72K
CSFN (no bidirectional)	$g_{\Lambda \rightarrow \Psi} = 1$; $W_{\Lambda \rightarrow \Psi}$ removed	70K
CSFN (no gates)	$g_{\Psi \rightarrow \Lambda} = g_{\Lambda \rightarrow \Psi} = 1$	69K
CSFN (shared decay)	Single MLP for both streams	72K
CSFN (full)	All mechanisms active, H=64	74K

Table 3: Baselines and ablations. All $\sim 74K$ parameters; 5 random seeds; mean $\pm$ std reported.

7.2 Task 1: Delayed Copy Task

Setup. Input $x_{1:T} \sim N(0, I_d)$, $d=10$, $T=100$. The model must output $y_t = x_{t-k}$ for delays $k \in \{5, 10, 20\}$. Loss is MSE over $t > k$. Batch size 32, Adam lr=1e-3, 500 epochs, 5 seeds. Delays $k=10$ and $k=20$ are designed to exceed the expressive stream's natural horizon and require the inner stream.

7.3 Task 2: Character-Level Language Modelling (PTB)

Setup. Standard Penn Treebank train/validation/test split. All models use H corresponding to $\sim 74K$ parameters. Metric: bits per character (BPC) on the test set. Sequence length 256, batch 64, Adam lr=1e-3, cosine annealing. Single seed (standard protocol). This task tests whether the cognitive architecture provides practical language modelling gains over standard recurrent baselines.

7.4 Task 3: Dual-Context Tracking

Setup. A novel diagnostic task designed to directly test the ability to simultaneously maintain two independent signal sources. The input at each step t is a concatenation of two independent random signals $[a_t; b_t]$, $a_t, b_t \sim N(0, I_5)$. The model must simultaneously output $y_{a,t} = a_{t-5}$ and $y_{b,t} = b_{t-10}$: the first source at a short delay and the second at a longer delay. Models with a single hidden state face a bottleneck: the two memory traces interfere. Models with two separated streams can in principle assign each trace to one stream. $d=10$, $T=100$, 5 seeds.

This task is designed specifically to discriminate architectures with explicit parallel processing capability. Neither a single-stream baseline nor an ablation without the bidirectional coupling should solve it as well as full CSFN.

7.5 Fatigue Analysis Protocol

After training full CSFN on Task 1 ($k=10$), we run inference on 100 held-out sequences and log $\phi_{\Lambda,t}$ and $\phi_{\Psi,t}$ at each timestep. We report: (i) mean fatigue level by stream across the sequence, (ii) fraction of neurons with $\phi > 0.5$ ("saturated") at each step, (iii) correlation between $\phi_{\Lambda,t}$ and $\phi_{\Psi,t}$ — high correlation would suggest the two streams are fatiguing in lockstep, undermining the independence claim. A healthy architecture should show decorrelated fatigue profiles between streams and temporal variation in saturation fraction.

8. Results and Analysis

Note: The following result tables are populated from preliminary runs (single seed) pending full 5-seed evaluation. Mean $\pm$ std will be reported in the final version. Entries marked [—] are awaiting computation.

8.1 Delayed Copy Task

Model	k=5 MSE	k=10 MSE	k=20 MSE
GRU (matched)	—	—	—

LSTM (matched)	—	—	—
Mamba-tiny	—	—	—
xLSTM-small	—	—	—
CSFN (no fatigue)	—	—	—
CSFN (no bidir.)	—	—	—
CSFN (no gates)	—	—	—
CSFN (shared decay)	—	—	—
CSFN (full)	—	—	—

Table 4: Delayed copy task. MSE ↓, mean ± std over 5 seeds. Bold = best per delay.

We expect single-state baselines (GRU, LSTM) to perform well at $k=5$ but degrade significantly at $k=20$, where the memory trace must survive over the full temporal horizon without interference from new inputs. The no-fatigue and no-bidirectional ablations reveal which mechanism contributes more to long-delay performance. The no-gates ablation isolates whether instability during training (rather than representational capacity) accounts for baseline shortfalls.

8.2 Character-Level PTB

Model	BPC (test) ↓
GRU (matched)	—
LSTM (matched)	—
Mamba-tiny	—
xLSTM-small	—
CSFN (full)	—

Table 5: Character-level PTB BPC. Single seed; standard train/val/test split.

8.3 Dual-Context Tracking

Model	MSE_a ($k=5$) ↓	MSE_b ($k=10$) ↓	Total MSE ↓
GRU (matched)	—	—	—
LSTM (matched)	—	—	—
CSFN (no bidir.)	—	—	—
CSFN (full)	—	—	—

Table 6: Dual-context tracking. MSE_a tracks source a at delay 5; MSE_b tracks source b at delay 10.

8.4 Fatigue Profile Analysis

Expected: (i) mean $\phi_\Psi < \text{mean } \phi_\Lambda$ across the sequence, reflecting the inner stream's slower activation dynamics; (ii) saturation fraction for Λ rises then falls as the model processes rapidly changing input, while Ψ saturates more gradually; (iii) Pearson correlation between $\phi_{\Lambda,t}$ and $\phi_{\Psi,t} < 0.4$, confirming the two streams fatigue independently. These results will be reported as time-series plots in the final version.

9. Limitations and Future Work

Quadratic hidden cost.

GRU cells and feedback projections scale as $O(H^2)$, matching standard GRUs but lagging behind linear SSMs such as Mamba and Griffin. Structured or low-rank feedback matrices (e.g., Monarch [18] factorisation) could reduce this without sacrificing expressivity.

Two fixed timescales.

The dual-stream design is manually specified. A natural extension would learn a mixture over N timescale streams via soft attention over a bank of GRU cells, analogous to mixture-of-experts. This would allow the network to discover how many timescales the task requires.

Scalar β for slow-stream decay.

The exponent $\beta=0.2$ enforcing slow forgetting in Ψ is a global hyperparameter. Learning β per-neuron (as a trained scalar) would allow heterogeneous forgetting rates within the inner stream, reflecting the biological reality that different neuron populations have different memory time constants.

No auxiliary cognitive loss.

CSFN is trained solely on the task loss. Adding an auxiliary loss that rewards the inner stream for predicting future inputs (predictive coding [11]) would give the inner stream an explicit function — anticipation — encouraging it to maintain richer long-horizon representations even when the task loss alone does not require it.

Fixed fatigue recovery rates γ .

While γ is learned, it is input-independent. A dynamic recovery rate — one that speeds up when the input changes rapidly and slows down during stable inputs — would better model the activity-dependent recovery observed in biological synapses. This could be implemented as a second MLP conditioned on the current input and previous fatigue level.

Evaluation scope.

The current experimental suite covers sequential signals and character-level text. Audio, video frame prediction, and multi-speaker dialogue (where inner and expressive streams have natural cognitive referents) remain important future benchmarks.

10. Conclusion

We presented the Cognitive Stream Fatigue Network, a recurrent architecture that decomposes hidden dynamics into an inner (conceptual) stream Ψ and an expressive stream Λ , coupled bidirectionally by two independent learned confidence gates and regulated by per-stream neural fatigue accumulators. Fatigue is modelled as an exponentially decaying activation trace per neuron, with learned per-neuron recovery rates; its effect is to prevent representational fixation and force diversity of activation across time — a functional property absent from all prior sequence models. Bidirectional coupling allows inner thought to guide surface expression while expression simultaneously feeds back into background reasoning, reflecting the observed bidirectionality of biological neural circuits.

The architecture is grounded in neuroscience (synaptic short-term depression), motivated by cognitive science (parallel inner and outer speech), and concretely implemented as a parameter-efficient PyTorch module. Three complementary evaluation tasks — delayed copy at multiple delays, character-level language modelling, and the novel dual-context tracking task — provide a discriminative experimental protocol for future comparison. We release the full reference implementation and hope CSFN serves as a useful building block for streaming, real-time, and cognitively-motivated sequence modelling systems.

References

- [1] Vaswani, A., et al. (2017). Attention is all you need. NeurIPS 30.
- [2] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780.
- [3] Cho, K., et al. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. EMNLP.
- [4] Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. IEEE Trans. Neural Networks, 5(2), 157–166.
- [5] Gu, A., & Dao, T. (2023). Mamba: Linear-time sequence modeling with selective state spaces. arXiv:2312.00752.
- [6] Gu, A., Goel, K., & Ré, C. (2021). Efficiently modeling long sequences with structured state spaces. ICLR.
- [7] Brown, T., et al. (2020). Language models are few-shot learners. NeurIPS 33.
- [8] Dai, Z., et al. (2019). Transformer-XL: Attentive language models beyond a fixed-length context. ACL.
- [9] Chung, J., Ahn, S., & Bengio, Y. (2017). Hierarchical multiscale recurrent neural networks. ICLR 2017. arXiv:1609.01704.
- [10] Koutnik, J., Greff, K., Gomez, F., & Schmidhuber, J. (2014). A clockwork RNN. ICML 2014.
- [11] Rao, R. P., & Ballard, D. H. (1999). Predictive coding in the visual cortex. Nature Neuroscience, 2(1), 79–87.
- [12] Lotter, W., Kreiman, G., & Cox, D. (2016). Deep predictive coding networks for video prediction and unsupervised learning. arXiv:1605.08104.
- [13] Zucker, R. S., & Regehr, W. G. (2002). Short-term synaptic plasticity. Annual Review of Physiology, 64, 355–405.

- [14] Beck, M., et al. (2024). xLSTM: Extended long short-term memory. arXiv:2405.04517.
- [15] De, S., et al. (2024). Griffin: Mixing gated linear recurrences with local attention for efficient language models. arXiv:2402.19427.
- [16] Peng, B., et al. (2023). RWKV: Reinventing RNNs for the transformer era. EMNLP 2023.
- [17] Sun, Y., et al. (2023). Retentive network: A successor to transformer for large language models. arXiv:2307.08621.
- [18] Dao, T., et al. (2022). Monarch: Expressive structured matrices for efficient and accurate training. ICML 2022.